Mk33: A Fully Specified Recipe and Reference Implementation for Training Long-Horizon Self-Improving Agents

Dulguun Enkhtsatsral
New York University Shanghai
de2318@nyu.edu

Abstract

AREX-2 shows that a 27B-parameter agent trained by imitation on long-horizon improvement trajectories can keep refining a solution for hours, and that this ability transfers to deep-research tasks. The paper describes its training recipe in prose and releases only evaluation code. Several components a reimplementation needs are left open: numeric thresholds for filtering environments and accepting trajectories, a definition of the "no-progress" turns excluded from the loss, the agent's tool set, and what happens when a multi-hour run exceeds the context window. We present Mk33, a complete specification of this recipe together with an open reference implementation. It contributes (1) a noise-aware environment filter, baseline-to-reference score normalization, benchmark decontamination, and an isolated, hash-checked scorer with a process audit; (2) Round Ledger Compaction (RLC), a context manager that keeps every round, including failures, visible in a structured ledger and freezes the context prefix between compactions, so each segment between compactions is exactly one training window; (3) Progress-Aware Loss Masking (PALM), deterministic rules for redundant, null, and polling turns plus provenance-based credit for turns that lead to score improvements; and (4) a full training configuration. The implementation includes tests that check, among other properties, that every context the agent sends to the model is reproduced exactly by the offline training replay. We define an evaluation protocol on the six AREX-2 benchmarks with seven ablations and process metrics that measure reflection and long-horizon execution directly. We also release a small CPU-only benchmark that runs the scaffold against any OpenAI-compatible model and report a first pilot run on it. This report specifies the method and protocol; results at the scale of the protocol are not yet available.

1 Introduction

An agent that can improve its own solution over many rounds converts extra test-time compute into better results. AREX-2 [12] frames this as self-improvement through test-time scaling and splits the gain into two capabilities: *reflection*, which sets the mean gain per productive round, and *long-horizon execution*, which sets how many rounds remain productive. Trained only by imitation on teacher trajectories from machine-learning engineering and competitive-programming environments, AREX-2 reports 81.8 on MLE-bench Lite and 70.7 on Frontier-CS, along with gains on deep-research benchmarks that provide no score feedback.

The result is useful but difficult to build on. The public repository contains evaluation runners only. The paper specifies its pipeline qualitatively: environments are kept when a baseline scores "well below the reference"; the loss skips "near-duplicate turns"; trajectories are accepted when the final score is "high." Two questions receive no answer: how a run that lasts up to 12 hours fits in a finite context window, and which teacher model generates the data.

This report does not claim a new capability. Its goal is to make the AREX-2 recipe reproducible and testable. For each component AREX-2 leaves open, we give a precise specification, a rationale, a configuration key, and an ablation that tests whether the choice matters. We also release a reference implementation, built as an extension of the AREX-2 repository, so that each specification can be checked against working code.

Contributions.

1. **Environment specification** (Section 3): baseline-to-reference normalization, a noise-aware headroom filter, benchmark decontamination, and scorer isolation with a process audit against reward hacking.
2. **Round Ledger Compaction** (Section 4): a context manager for multi-hour runs that keeps a structured record of every round, freezes the context prefix between compactions, and is shared verbatim by the agent and the training data pipeline.
3. **Progress-Aware Loss Masking** (Section 5): formal rules for zero-weight turns and a provenance set of full-credit turns, computed deterministically from trajectory logs.
4. **A complete recipe and reference implementation** (Sections 6 and 7): acceptance rules, every training hyperparameter, configuration files for each ablation, and tests for the properties the method depends on.
5. **An evaluation protocol** (Section 8): seven ablations and five process metrics that measure the two capabilities AREX-2 names, rather than final scores alone.
6. **A CPU pilot benchmark** (Section 9): four seeded environments that run the full scaffold against any OpenAI-compatible model, with a first pilot run.

2 Background and Gap Analysis

The AREX-2 framework. In round t the agent submits a solution y_t and receives feedback f_t: a score plus logs, errors, and timings. The expected total improvement over T rounds is

$$\mathbb{E}[\Delta_{\text{total}}] \;=\; \sum_{t=1}^{T} r_t \;\approx\; \bar{r} \cdot T^{*}, \tag{1}$$

where $\bar{r}$ is the mean gain per productive round and T^{*} is the number of productive rounds. The AREX-2 pipeline has four stages: (i) build environments from GitHub repositories and online judges; (ii) generate long teacher trajectories that keep every round, including regressions; (iii) accept whole trajectories by final score and process conformance; and (iv) fine-tune Qwen3.8-27B [13] with loss only on turns that make progress, mixed with unchanged deep-research data from the AREX recipe [6].

What is unspecified. Table 1 lists each component that the paper or its repository leaves open, and where this report addresses it. We add one further check that AREX-2 does not run. Its claim that gains transfer across domains rests on holding the deep-research data fixed. That design does not separate the effect of iterative trajectory structure from the effect of additional agentic training tokens. Ablation A7 (Section 8) adds a token-matched control.

3 Environments

Mk33 uses the same two environment sources as AREX-2. For *ML repositories*, a candidate must have a permissive license, a runnable entry point, a quantitative metric already computed in its code, and a full run under 2 GPU-hours on one 80 GB GPU. A teacher model reads a digest of the repository and drafts a task statement asking for the metric to be improved, a deterministic held-out split, a scoring script, a baseline command, and a reference command. For *online-judge problems*, we prefer graded scoring: optimization problems scored against the best known value, or problems with many hidden tests scored by the fraction passed.

Table 1: Components of the AREX-2 recipe that are unspecified, and where Mk33 specifies them.

Component	Stated in AREX-2	Missing	Here
Environment filter	Baseline "well below the reference"	Numeric margin; score noise	§3.2
Score scale	Graded scores "where possible"	Normalization across metrics	§3.1
Contamination	Not discussed	Overlap with evaluation tasks	§3.3
Process conformance	Process must conform	Concrete checks	§3.4
Agent scaffold	Tool calls, submission, feedback	Tool set; scaffold code	§4.1
Context management	Failures stay in context; 12 h runs	Behavior on overflow	§4
Loss masking	No loss on polling, empty calls, near-duplicates	Detection rules; "progress"	§5
Acceptance	Best score above a relative threshold	Values; minimum rounds	§6.1
Teacher model	"A teacher model"	Identity; sensitivity	§6.2
Training	Base model; data mix	All hyperparameters	§6.3
Released code	Evaluation only	Synthesis, scaffold, training	§7

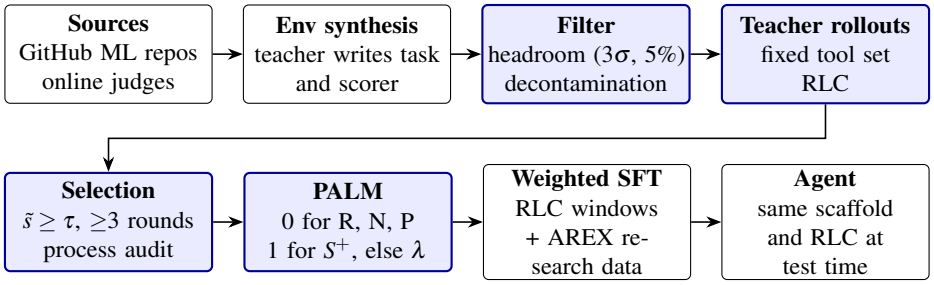


Figure 1: The Mk33 pipeline. Shaded stages contain components that are new relative to AREX-2; the others follow AREX-2 and are now fully specified. The same scaffold and context manager are used for teacher rollouts, for building training windows, and at test time.

3.1 Normalized score

Raw metrics differ in scale and direction. For environment e with baseline score b_e (a simple solution such as a default configuration, majority class, or brute force) and reference score $s_{\mathrm{ref},e}$ (the mean of k seeded runs), every score is mapped to

$$\tilde{s} \;=\; \frac{s - b_e}{s_{\mathrm{ref},e} - b_e}, \tag{2}$$

with all three quantities negated when lower is better. The baseline maps to 0 and the reference to 1. Run-to-run noise on this scale is $\varepsilon_e = \sigma_e / |s_{\mathrm{ref},e} - b_e|$, where σ_e is the standard deviation of the reference runs; ε_e is used below to decide whether a change in score is real.

3.2 Noise-aware headroom filter

AREX-2 requires the baseline to score "well below" the reference. We run the reference $k = 3$ times and keep an environment only if

$$s_{\mathrm{ref},e} - b_e \;\geq\; \max\bigl(3\,\sigma_e,\; 0.05\,|s_{\mathrm{ref},e}|\bigr). \tag{3}$$

The $3\sigma_e$ term ensures that improvement on the normalized scale is a real signal rather than seed noise. The 5% term rejects environments whose headroom is statistically significant but too small to support many productive rounds. For judge problems scored by pass fraction we instead require $b_e \leq 0.6$ and $s_{\mathrm{ref},e} \geq 0.95$. Both margins are swept in ablation A4.

3.3 Decontamination

GitHub ML repositories can overlap with Kaggle competitions in MLE-bench [2], and judge problems can overlap with Frontier-CS [8]. An environment is dropped if, against any evaluation task, it shares a dataset (normalized URL, normalized name, or file hash), shares a competition or problem identifier, or has 13-gram overlap of at least 50% between task statements, following the n-gram check of Brown et al. [1]. We report the number of environments each rule removes. Ablation A6 measures how much of the final score depends on this step.

3.4 Scorer isolation and process audit

AREX-2 accepts trajectories only if the process conforms to the task requirements, without stating how this is checked. Verifiable environments invite reward hacking [15], so we enforce conformance by construction and then audit it.

1. The held-out split and scoring script live in a scorer directory that is never mounted in the agent's sandbox. The agent sees only a public development split and a `submit` tool.
2. Before every run the scorer verifies the SHA-256 hash of its own script and scores a frozen copy of the solution. In container mode the solution is mounted read-only and the scorer has no network access.
3. For judge problems, 10% of hidden tests are canaries generated after the trajectory is collected. A solution whose pass rate on regular tests exceeds its pass rate on canaries by more than 0.2 is flagged as overfit.
4. Rule checks flag reads of paths outside the workspace or inside the scorer directory, hidden-test outputs embedded as literals in the final solution (one output of at least 32 characters, or three of at least 8), and fetches of the source repository's reference solution.
5. An optional LLM auditor reviews the final diff for metric gaming, such as changing the evaluation split.

A trajectory flagged by any of steps 3 to 5 is rejected.

4 Agent Scaffold and Round Ledger Compaction

4.1 Tool set

AREX-2 does not list its tools. We fix one set for all environments so that trajectories share an action space (Table 2). The blocking `wait_job` tool is a deliberate change: in AREX-2, polling turns are generated and then masked out of the loss; a blocking wait removes most of them at the source, which saves context and wall-clock time.

4.2 The overflow problem

A 12-hour run with hundreds of tool calls produces far more tokens than a context window holds. AREX-2 states that failures and regressions remain in the context, but not what happens when it is full. Truncating the oldest turns removes the early failures the model is meant to learn from; free-form summarization loses scores and error signatures.

Table 2: The fixed tool set.

Tool	Behavior
`bash`	Shell command in the sandbox; 10-minute timeout; output truncated to about 8K tokens, keeping head and tail.
`read_file`, `edit_file`	Inspect and change files. Edits require a unique match and return a unified diff, which PALM uses for provenance.
`run_job`, `wait_job`	Start a long command; block until it finishes or a timeout (at most 30 minutes) passes, returning only new log lines.
`web_search`, `web_fetch`	Find papers, documentation, and libraries. Disabled where a benchmark forbids it.
`submit`	Score the current solution. Requires a `hypothesis` and a one-line `lesson`. Ends the round.
`answer`	Final answer for research tasks.

Algorithm 1 Round Ledger Compaction: building the context for the next turn

Require: window W, threshold θ, rounds kept K, ledger cap L; current round r

if prefix is unset **then** prefix $\leftarrow$ RENDER(pinned, ledger)
$C \leftarrow$ prefix $\|$ visible turns
if $|C| \leq \theta W$ **then return** C ▷ no event: context only grew
$c' \leftarrow r - K + 1$
if $c' \leq c$ **and** $|\text{ledger}| \leq L$ **and** $|C| \leq W$ **then return** C ▷ nothing to compact yet
$c \leftarrow \max(c, c')$ ▷ turns from rounds $< c$ are now ledger-only
if $|\text{ledger}| > L$ **then** merge oldest non-best entries per strategy family
prefix $\leftarrow$ RENDER(pinned, ledger) ▷ includes a note that rounds $1..c-1$ were compacted
$C \leftarrow$ prefix $\|$ visible turns
while $|C| > W$ **and** more than one visible turn **do** drop the oldest visible turn
record a compaction event; **return** C

4.3 Round ledger

The scaffold keeps a *round ledger* with one entry per submission. Most fields are filled automatically: round number, elapsed time, a diff summary of the round's edits, raw score, normalized score, change relative to the best so far, and the first error line of a failed run. The agent writes only two fields, the `hypothesis` and `lesson` arguments of `submit`. Each new entry is also appended to the `submit` observation, so the model sees it immediately. When the ledger exceeds a token cap, its oldest non-best entries are merged into one line per strategy family, recording their rounds, best score, number of failures, and recent lessons. No round is deleted, so the AREX-2 principle that failures stay visible holds for runs of any length. For research tasks without a scorer, entries record a candidate answer, a confidence, and evidence URLs instead of a score.

4.4 Compaction with a frozen prefix

The context consists of a *prefix* (system prompt, task statement, skills, current best, and the rendered ledger) followed by the verbatim turns of recent rounds. Algorithm 1 gives the procedure. The prefix is rendered at the start of a run and re-rendered only when a compaction occurs. Between compactions the context therefore grows only by appending turns. This has two consequences. First, key-value caches remain valid across turns. Second, and more important for training, every segment between two compactions is a single sequence that covers all of its turns, which makes the training windows of Section 4.5 exact rather than approximate.

Defaults are $W = 128$K tokens, $\theta = 0.75$, $K = 2$, and $L = 16$K tokens. The final loop is a guard for a

single round that alone exceeds the window.

4.5 Identical contexts at training and test time

A model trained on uncompacted trajectories but run with compaction sees a context distribution it was never trained on. We avoid this by construction: one context-building component is used by the live agent and by the offline training replay. Tool-call identifiers are deterministic (`call_`*i* for turn *i*), so the replay emits the same messages byte for byte. The teacher runs with RLC during collection, so its decisions are conditioned on the same views. To build training data, each accepted trajectory is replayed; every compaction event starts a new window whose prefix is the compacted context at that moment, and the turns that follow are appended until the next event. Loss is applied only to assistant turns inside the window, weighted by PALM. The reference implementation tests this property directly (Section 7).

5 Progress-Aware Loss Masking

AREX-2 applies loss to "decisions that move the solution forward" and none to "repeated polling, calls that return no new observation, and near-duplicate turns," without saying how these are detected. PALM turns these descriptions into deterministic rules over the trajectory log, so masks can be reproduced and audited.

Setup. A trajectory has assistant turns $a_1, \ldots, a_N$, each consisting of reasoning and one tool call, followed by an observation o_i. After each turn the scaffold records a hash h_i of the workspace. Let B_t be the best normalized score after round t, with $B_0 = 0$ (the baseline). A submission at round t is *improving* if $\tilde{s}_t > B_{t-1} + \varepsilon_e$.

5.1 Zero-weight turns

A turn receives weight 0 if any of the following holds.

- **Redundant**, $R(i)$: some earlier turn j in the last $W_R = 20$ turns has a near-identical canonical action (tool name and arguments with whitespace normalized; Jaccard similarity of word 5-gram shingles ≥ 0.9), and the workspace has not changed since turn j, including by turn i itself. Re-running a training script after an edit is therefore a new experiment, not a duplicate. Status checks are excluded from R and handled by P, because a repeated status check can return new information.
- **Null observation**, $N(i)$: after normalization (removing timestamps, process IDs, durations, ANSI codes, and progress-bar lines) the observation is empty with the workspace unchanged, or identical to the last observation from the same tool on the same target with the workspace unchanged.
- **Polling**, $P(i)$: a status-only call (for example `wait_job` returning on timeout, `ps`, `nvidia-smi`, or `tail` on a log) that adds no new log lines since the last status call on the same target.

System messages, observations, and retrieved documents also receive no loss, as in AREX-2.

5.2 Full-credit turns

A turn joins the provenance set S^+ if any of the following holds.

1. *Surviving edit*: it is an `edit_file` turn and at least one of its diff hunks appears line for line (ignoring blank lines and trailing whitespace) in the solution scored at a later improving submission. A deletion-only hunk survives if the removed block is still absent. This approximates `git blame` without requiring repository history.

2. *Improving submission*: it is the `submit` call of an improving round.

3. *Supporting evidence*: it is a read, search, fetch, or inspection turn within 50 turns before a turn from rule 1 or 2, and the later turn uses it: the two share a file path, URL, or identifier, or an 8-gram of the observation appears in the later turn's reasoning. Paths and identifiers that occur in the task statement are excluded, since every file the task names would otherwise qualify. Only one hop is followed.

4. *Recovery*: it is the first edit after a regression ($\tilde{s}_t < B_{t-1} - \varepsilon_e$) or a failed submission, it touches the file named in the error if one is named, and the agent regains at least B_{t-1} within $M = 3$ rounds. This credits diagnosis and repair even when the repaired code is later replaced.

5.3 Weights and loss

All remaining turns receive weight λ. These are plausible steps that did not lead to a gain, such as a reasonable experiment that failed. The AREX-2 description implies $\lambda = 0$ for some of them, yet trajectories are accepted without filtering rounds so that the model learns to explore. We therefore treat λ as an open question, default to $\lambda = 0.5$, and sweep it in ablation A1:

$$w_i = \begin{cases} 0 & \text{if } R(i) \vee N(i) \vee P(i), \\ 1 & \text{if } i \in S^+, \\ \lambda & \text{otherwise.} \end{cases} \tag{4}$$

The loss is a weighted token-level cross-entropy normalized by the weighted token count, so that long trajectories do not dominate a batch:

$$\mathscr{L}(\theta) \;=\; -\frac{\sum_i w_i \sum_{k \in a_i} \log p_\theta(x_k \mid x_{<k})}{\sum_i w_i \, |a_i|}. \tag{5}$$

Reasoning tokens inside a turn share the turn's weight.

5.4 Checking the rules

Heuristic rules can be wrong, so we measure them before training. Two annotators label 500 turns sampled from 50 trajectories as *no progress*, *neutral*, or *progress*. We report precision and recall of the zero-weight rules and of S^+ against these labels, Cohen's κ between the rules and the adjudicated labels, and κ between the two annotators. If the rules agree with the labels less than the annotators agree with each other, thresholds are revised before any training run.

6 Trajectory Selection and Training Recipe

6.1 Acceptance rule

As in AREX-2, whole trajectories are accepted or rejected and individual rounds are never filtered. A trajectory is accepted if (i) its best normalized score satisfies $\tilde{s}_{\text{best}} \geq \tau_d$, with $\tau_{\text{ML}} = 0.9$ and $\tau_{\text{judge}} = 0.95$; (ii) it has at least 3 scored rounds; (iii) at least one round after the first is improving, so every accepted trajectory shows iteration; and (iv) the process audit raised no flag. At most two trajectories are kept per environment, preferring higher best scores, so that easy environments do not dominate. Ablation A3 sweeps $\tau \in \{0.7, 0.9, 1.0\}$.

Table 3: Planned data scale and training configuration (proposed defaults, not tuned values).

Item	Value
Environments after filtering	2,000 (1,200 ML, 800 judge)
Teacher rollouts	4 per environment, temperature 0.7
Rollout budget	12 h (ML) or 5 h (judge); at most 400 tool calls
Deep-research data	AREX recipe data, unchanged; default token ratio 1:1
Base model	Qwen3.8-27B (as in AREX-2)
Method	Full-parameter SFT with PALM weights; no reinforcement learning
Context	128K tokens, RLC windows (Section 4.5)
Optimizer	AdamW, $\beta = (0.9, 0.95)$, weight decay 0.1
Learning rate	Peak 10^{-5}, 3% linear warmup, cosine decay to 10^{-6}
Epochs; global batch	2; 64 windows
Gradient clipping	1.0
Precision; parallelism	bf16; ZeRO-3 with sequence parallelism
Seeds	3 for the main model, 1 per ablation

6.2 Teacher model

AREX-2 does not name its teacher, although results may depend heavily on it. We select the teacher with a pilot: each candidate runs on 50 held-out environments, and we choose the one with the most accepted trajectories per GPU-hour. We prefer an open-weight teacher so that trajectories can be redistributed under its license. The teacher and version will be reported with results. Ablation A5 retrains with the runner-up teacher.

6.3 Data and hyperparameters

Table 3 lists the planned data scale and the training configuration. These are proposed defaults chosen to match common practice for full-parameter supervised fine-tuning at this scale; they will be tuned on a validation split of 100 environments, and any changes will be reported.

7 Reference Implementation

We release the implementation as a standalone Python package; the benchmarks in Section 8 are run with the AREX-2 evaluation harness, released separately by its authors, so that scores are directly comparable. The package has six modules: `envs` (normalization, filter, decontamination, scorer, audit, synthesis), `scaffold` (tools, sandboxes, ledger, RLC, shared context builder, agent loop), `palm` (rules, provenance, weights, validation), `selection`, `train` (window construction and the weighted loss of Equation (5)), and `metrics`. A command-line tool runs each pipeline stage, and every threshold in this report is a configuration value. Each ablation in Table 4 is a configuration file that inherits the defaults and overrides only what it changes.

The test suite checks the properties the method depends on, including: normalization for both metric directions; each filter and decontamination rule; that the scorer refuses to run when its script hash changes; each audit rule; that ledger compaction never drops a round and never merges the best entry; that RLC contexts are append-only between events; each PALM rule, including that a re-run after an edit is not redundant and that a status check with new lines is not polling; and the acceptance rule and process metrics. An end-to-end test runs a scripted agent on a small environment with a context window small enough to force compaction, then computes PALM weights, applies the acceptance rule, builds training windows, and asserts that *every*

Table 4: Ablations. Each changes one component and keeps the rest at the defaults.

ID	Question	Variants
A1	Does PALM beat simpler masks?	All assistant tokens; AREX-style mask (zero-weight rules only, $\lambda = 1$); LLM-judge mask; PALM with $\lambda \in \{0, 0.25, 0.5, 1\}$
A2	Does RLC matter, and does consistency?	Truncation; free-form summary; RLC at test time only; RLC at training and test time
A3	How strict should acceptance be?	$\tau \in \{0.7, 0.9, 1.0\}$
A4	Does the headroom filter matter?	No filter; 5% margin only; 3σ only; both
A5	How much does the teacher matter?	Selected teacher; runner-up teacher
A6	How much of the gain is contamination?	With and without decontamination, evaluated on both
A7	Is transfer caused by iteration?	Token-matched single-attempt trajectories from the same environments

context the agent sent to the model is a prefix of the training window that covers that turn. This is the property that Section 4.5 relies on. The weighted-loss and chat-template code has not yet been exercised on GPUs.

8 Evaluation Protocol

Benchmarks and budgets. We use the six AREX-2 benchmarks and budgets so that results are directly comparable: MLE-bench Lite [2] (Any Medal; 12 h per task with skills in context), the Frontier-CS Agent Track [8] (188 tasks; 5 h per task), and BrowseComp [16], HLE [11], GAIA [9], and DeepSearchQA [4] (300 inner turns and 1,500 turns overall). Each configuration runs three times; we report the mean and a 95% bootstrap confidence interval. For reference, AREX-2 reports 81.8, 70.7, 84.0, 52.6, 92.2, and 93.8 on these benchmarks, in the order listed.

Systems. (i) *Base*: Qwen3.8-27B in our scaffold with RLC, untrained. (ii) *AREX-2*: the public weights, run both in our scaffold and in the AREX-2 harness, which separates the effect of the model from the effect of the scaffold. (iii) *Mk33*: the full method.

Process metrics. Final scores do not show which capability improved. Following Equation (1), we also report: *productive rounds* T^*, the number of rounds before the first run of five rounds without an improving submission; *mean gain* $\bar{r}$, the mean normalized gain over improving rounds; *best-so-far AUC*, the area under the normalized best-so-far curve over the time budget; *recovery rate*, the share of regressions followed by a new best within three rounds; and *wasted-turn share*, the share of test-time turns flagged by *R*, *N*, or *P*.

Ablations. Each ablation changes one component (Table 4). A7 is the central test of the AREX-2 transfer claim: if single-attempt trajectories matched on token count produce the same deep-research gains, those gains come from additional agentic training rather than from learning to iterate. Following AREX-2, we also plot best-so-far score against time on Frontier-CS (with feedback) and against turn budget on BrowseComp (without feedback), and add these curves for each A2 variant, since context management should matter most late in long runs.

9 Pilot Benchmark

The protocol in Section 8 needs GPUs and multi-hour runs. To test the scaffold end to end before that, the implementation includes a small benchmark that runs on one CPU with any OpenAI-compatible model. It

Table 5: Pilot benchmark environments (seed 0). Raw scores from the isolated scorer; the reference is run with three seeds. Normalized scores map the baseline to 0 and the reference mean to 1.

Environment	Metric	Baseline	Reference runs
Regression	RMSE ↓	4.970	2.616, 2.614, 2.642
Sentiment	accuracy ↑	0.511	0.653, 0.657, 0.659
Forecast	MASE ↓	4.239	1.041, 1.019, 0.984
Knapsack	optimality gap ↓	0.321	0.0052, 0.0063, 0.0048

reuses the agent loop, tool set, isolated scorer, headroom filter, process audit, and process metrics without changes.

Environments. Four environments are generated from a fixed seed and use only the Python standard library (Table 5): nonlinear regression (Friedman #1, RMSE), sentiment classification of synthetic reviews with negation and 3% label noise (accuracy), forecasting of 30 daily series with trend, weekly seasonality, and level shifts (MASE), and 24 hidden 0/1 knapsack instances (mean optimality gap). The agent sees a task statement, a weak baseline solution, and the public split; the held-out split, scorer, and reference solutions are kept in the isolated scorer directory. Baseline and reference scores are measured by running the real scorer, and all four environments pass the headroom filter of Section 3. Knapsack has a known optimum, so its normalized score is capped near 1.02; the other three leave room above the reference.

Design. The benchmark runs an inference-time version of ablation A2: with the model fixed, it compares RLC against truncation of the oldest turns, using a 24K-token window so that long runs must be compacted. Both modes run on the same environments, seeds, and budgets, and the report gives paired differences with seeded bootstrap 95% intervals. Runs are resumable, and model calls can be rate-limited for free API tiers.

First run. We ran one hosted model through a free API tier on the knapsack environment with 25 tool calls and 8 rounds per run. The tier's limit of 50 requests per day allowed one pair before the remaining runs were stopped. The truncation run submitted one round and reached a normalized score of 1.017, the optimum. The RLC run wrote and tested an exact dynamic-programming solver but used its 25 tool calls before submitting, so it scored 0. Until the first compaction the two modes send nearly the same context, and the RLC ledger is empty before the first submission, so this pair carries no evidence about RLC. It shows that the pipeline works end to end with a real model, that runs vary widely, and that 25 tool calls is too tight a budget. Earlier attempts with small local models (3B to 8B parameters on a laptop CPU) did not finish: a reasoning model exceeded the per-call timeout, and a 3B model described edits instead of calling the tools. The full grid (four environments, both modes, three repeats) remains to be run.

10 Related Work

Iterative refinement. Self-Refine [7] and Reflexion [14] showed that language models can improve outputs using their own feedback or verbal reflection, over a few rounds and mainly at inference time. AREX-2 and this work train for refinement over hundreds of rounds.

Machine-learning engineering and coding agents. MLE-bench [2] measures agents on Kaggle-style tasks. AIDE [5] frames machine-learning engineering as tree search over code. SWE-agent [17] shows that

the design of the agent-computer interface strongly affects performance, which motivates fixing the tool set in Section 4.1.

Agent fine-tuning. AgentTuning [18] and FireAct [3] fine-tune language models on agent trajectories. PALM additionally weights assistant turns by whether they contributed to progress, using rules computed from the trajectory log.

Context management. MemGPT [10] pages information between a bounded context and external memory. RLC is narrower: its memory has a fixed schema tied to submission rounds, the prefix is frozen between compactions, and the model is trained on exactly the compacted views it sees at test time.

Reward hacking. Optimizing a measurable proxy can diverge from the intended objective [15]. Scorer isolation, canary tests, and the audit in Section 3.4 address this in verifiable environments.

11 Limitations

- **No results at scale.** This report specifies the method, implementation, and protocol. The only measurements are one pilot pair on the CPU benchmark (Section 9), which tests the pipeline rather than the method. Whether each component helps is an empirical question that the ablations are designed to answer.
- **Heuristic thresholds.** The Jaccard cutoff, the windows W_R and M, and the acceptance thresholds are reasoned defaults rather than tuned values. The human-label check and ablations A1, A3, and A4 are intended to catch poor choices.
- **Incomplete provenance.** S^+ credits surviving edits and evidence one hop away. It misses insights that shape later decisions without shared tokens, such as learning that an approach is a dead end; such turns receive weight λ.
- **Teacher dependence.** Like AREX-2, the method imitates a teacher and can exceed the teacher's strategies only through generalization. A5 measures sensitivity but does not remove the dependence.
- **Incomplete implementation.** The free-form summary baseline of A2, the repository crawler, canary-test generation, and the LLM-judge prompt for A1 are not yet implemented.
- **Cost.** Collecting 8,000 rollouts of up to 12 hours each is expensive. Total GPU-hours and API cost will be reported with results.

12 Conclusion

AREX-2 shows that training on long improvement trajectories is a promising route to agents that improve with more time. Building on that result requires a recipe others can run. Mk33 provides one: explicit environment filters and thresholds, a context manager that keeps failures visible for runs of any length and yields exact training windows, deterministic progress-aware loss masking, and a complete training configuration, each paired with an ablation and implemented in open code.

Code Availability

The reference implementation, configuration files, and tests are available at https://github.com/dulguuntsatsralt-cell/Mk33.

Acknowledgments

This work builds on the AREX-2 paper and its open-source evaluation suite. The author used an AI assistant to help draft text and code; the author reviewed the content and is responsible for it.

References

[1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, 2020.

[2] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*, 2025.

[3] Baian Chen, Chang Shu, Ehsan Shareghi, Nigel Collier, Karthik Narasimhan, and Shunyu Yao. FireAct: Toward language agent fine-tuning. *arXiv preprint arXiv:2310.05915*, 2023.

[4] Nikita Gupta, Riju Chatterjee, Lukas Haas, Connie Tao, Andrew Wang, Chang Liu, Hidekazu Oiwa, Elena Gribovskaya, Jan Ackermann, John Blitzer, Sasha Goldshtein, and Dipanjan Das. DeepSearchQA: Bridging the comprehensiveness gap for deep research agents. *arXiv preprint arXiv:2601.20975*, 2026.

[5] Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. AIDE: AI-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138*, 2025.

[6] Shuqi Lu, Chaofan Li, Kun Luo, Zhang Zhang, Hui Wang, Hongwang Xiao, Lei Xiong, Jiahao Wang, Sen Wang, Xiyan Jiang, et al. AREX: Towards a recursively self-improving agent for deep research. *arXiv preprint arXiv:2607.21461*, 2026.

[7] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023.

[8] Qiuyang Mang, Wenhao Chai, Zhifei Li, Huanzhi Mao, Shang Zhou, Alexander Du, Hanchen Li, Shu Liu, Edwin Chen, Yichuan Wang, et al. FrontierCS: Evolving challenges for evolving intelligence. *arXiv preprint arXiv:2512.15699*, 2025.

[9] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general AI assistants. *arXiv preprint arXiv:2311.12983*, 2023.

[10] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[11] Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh Zhang, Chen Bo Calvin Zhang, Mohamed Shaaban, John Ling, Sean Shi, et al. Humanity's last exam. *arXiv preprint arXiv:2501.14249*, 2025.

[12] Hongjin Qian, Chaofan Li, Kun Luo, Wenqing Wei, Jianlyu Chen, Shuqi Lu, Yuyang Hu, Hongwang Xiao, Hui Wang, Chaozhuo Li, Qiwei Ye, Zhicheng Dou, Defu Lian, and Zheng Liu. AREX-2: Advancing self-improving agents through long-horizon reflective tasks. *arXiv preprint arXiv:2609.38288*, 2026.

[13] Qwen Team. Qwen3.8. https://github.com/QwenLM/Qwen3.8, 2026.

[14] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

[15] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems*, 2022.

[16] Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. BrowseComp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025.

[17] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.

[18] Aohan Zeng, Mingdao Liu, Rui Lu, Bowen Wang, Xiao Liu, Yuxiao Dong, and Jie Tang. AgentTuning: Enabling generalized agent abilities for LLMs. *arXiv preprint arXiv:2310.12823*, 2023.

A Default Configuration

The default configuration of the reference implementation (`configs/mk33/default.yaml`), abridged to the keys discussed in this report:

```
filter:    {sigma_k: 3.0, rel_margin: 0.05, min_reference_runs: 3,
            judge_max_baseline: 0.6, judge_min_reference: 0.95}
decontam:  {enabled: true, n: 13, threshold: 0.5}
audit:     {literal_min_len: 32, literal_short_len: 8,
            literal_short_count: 3, canary_gap: 0.2}
tools:     {bash_timeout_s: 600, max_output_tokens: 8000, max_wait_s: 1800}
rlc:       {mode: rlc, window: 128000, theta: 0.75, keep_rounds: 2,
            ledger_cap: 16000}
budget:    {ml: {max_tool_calls: 400, max_wall_s: 43200},
            judge: {max_tool_calls: 400, max_wall_s: 18000}}
selection: {tau: {ml: 0.9, judge: 0.95}, min_rounds: 3,
            require_iteration: true, max_per_env: 2}
palm:      {mode: palm, lam: 0.5, window: 20, jaccard: 0.9, m_rounds: 3,
            evidence_window: 50, ngram: 8}
sft:       {model: Qwen/Qwen3.8-27B, max_len: 131072, learning_rate: 1.0e-5,
            min_lr_ratio: 0.1, warmup_ratio: 0.03, weight_decay: 0.1,
            adam_beta1: 0.9, adam_beta2: 0.95, epochs: 2, global_batch: 64,
            max_grad_norm: 1.0, bf16: true}
```

B Data Formats

- **Environment specification**: task statement, workspace directory, scorer command (with `{solution}` and `{scorer_dir}` placeholders), script hash, metric direction, baseline score, reference scores, and dataset URLs, hashes, and problem identifiers for decontamination. The scorer prints `{"score": <float>}` as its last line of output.
- **Trajectory**: one JSON object per line. Each turn records reasoning, one tool call, the observation, the workspace hash, the diff for edits, and, for submissions, the score, normalized score, hypothesis, lesson, error signature, and a snapshot of the solution's text files.
- **Training window**: chat messages with tool calls, and one weight per message: the PALM weight for assistant targets and null for context. Deep-research data is converted to the same format with weight 1 on assistant messages.